
MARR: Module-Adaptive Residual Reconstruction for Low-Bit Post-Training Quantization

Le Su

School of Intelligent Systems Engineering
Shenzhen Campus of Sun Yat-sen University
Shenzhen, Guangdong 518107, China
sule@mail2.sysu.edu.cn

Xing Luo

Department of Frontier Research
Peng Cheng Laboratory
Shenzhen, Guangdong 518055, China
luox@pcl.ac.cn

Zhi Jin*

School of Intelligent Systems Engineering
Shenzhen Campus of Sun Yat-sen University
Shenzhen, Guangdong 518107, China
jinzh26@mail.sysu.edu.cn

Abstract

Recently, residual reconstruction-based model quantization methods have achieved promising performance in low-bit post-training quantization (PTQ) by introducing cross-layer residuals to reduce error accumulated from previous layers. However, these residuals may also introduce additional bias arising from the Hessian-approximation (HA) assumption underlying reconstruction-based PTQ, leading to suboptimal quantization performance. In this work, we analyze that multiplying the residual term by a scaling coefficient provides a direct way to mitigate the HA bias associated with residual strength, while preserving accumulated-error correction. More importantly, we observe that this trade-off is module-dependent, making a single global residual strength insufficient to balance effective correction and residual-related bias across modules. Based on these observations, we propose Module-Adaptive Residual Reconstruction (MARR), which assigns a module-specific scaling coefficient to adaptively balance accumulated-error correction and residual-related HA bias for each module. To avoid expensive per-module coefficient search and obtain a stable coefficient estimate, we design a Proportional-Integral-Derivative (PID)-based adaptive update strategy that uses reconstruction error as feedback to progressively refine this coefficient. Experiments on several typical large language models (LLMs) and vision transformers (ViTs) demonstrate the effectiveness of MARR under low-bit quantization (≤ 4 -bit), achieving up to 20.2% performance gains on LLMs and up to 4.6% relative gains on ViTs over the residual reconstruction state-of-the-art methods. Code will be made publicly available upon acceptance.

1 Introduction

Large language models (LLMs) [36, 41] and vision transformers (ViTs) [23, 9] have achieved remarkable progress across diverse tasks. However, their large model sizes and dense operations incur substantial memory, bandwidth, and inference overhead. To improve deployment efficiency, existing studies either compress pre-trained models through pruning [34, 6], knowledge distillation [45, 13],

*Corresponding author.

low-rank decomposition [42], and quantization [12, 46], or directly design lightweight architectures [21] and efficient operators [29, 15]. Among them, model quantization is deployment-friendly since it reduces the numerical precision of model while largely preserving the original architecture. Therefore, it is a plug-and-play compression technique compatible with existing inference pipelines and hardware kernels [11]. Moreover, pushing quantization below conventional 8-bit settings is increasingly important, as low-bit quantization can further reduce memory and bandwidth costs, allowing for larger models, longer sequences, or higher throughput under the same hardware budget [12]. Therefore, how to push models to lower bit-widths while preserving their performance remains a valuable and challenging problem.

Existing quantization methods are broadly divided into quantization-aware training (QAT) [24] and post-training quantization (PTQ) [18, 37]. Compared with QAT, PTQ avoids additional training or finetuning, making it more practical for extremely large pre-trained models. Current PTQ methods can be roughly grouped into quantization parameter-based methods [47], redistribution-based methods [40, 3], and reconstruction-based methods [11, 19]. Among them, reconstruction-based methods have attracted increasing attention for large models, since they efficiently reduce quantization error by adjusting weights while maintaining low quantization overhead through closed-form updates. A representative basic reconstruction method is GPTQ [11], which extends Optimal Brain Surgeon (OBS) [14] and Optimal Brain Quantization (OBQ) [10] by applying Hessian-based column-wise updates for each layer. However, under low-bit quantization, quantization errors from preceding layers accumulate across the network, making local-only basic reconstruction insufficient. To address this issue, recent residual reconstruction-based methods (GPTAQ [19] and ResComp [17]) account for inter-layer activation dependencies by introducing a residual between full-precision (FP) and quantized (Quant) activation, which improves low-bit quantization performance.

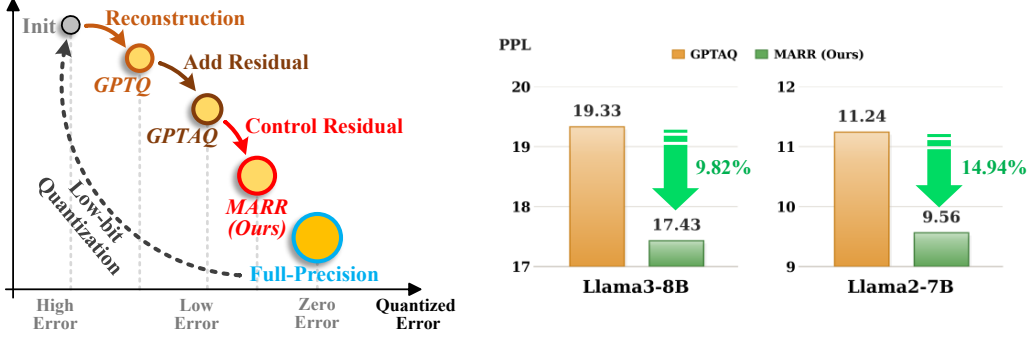
This cross-layer residual is beneficial for reducing accumulated quantization errors; however, it may also introduce additional Hessian-approximation (HA) bias. Such a two-sided effect makes the overall reconstruction suboptimal when overly strong residual correction is applied. In this work, to address this issue, we introduce a scaling coefficient to modulate the residual term, as the HA bias is directly affected by the strength of the residual correction. By doing so, residual reconstruction can preserve effective cross-layer error compensation while reducing the adverse effect of excessive residual strength. Moreover, the desired modulation is inherently module-dependent: the same residual strength can lead to different reconstruction behaviors across modules. This module-wise heterogeneity makes a single global coefficient insufficient. Accordingly, we propose Module-Adaptive Residual Reconstruction (MARR), which assigns a module-specific scaling coefficient to the residual term of each module. By directly scaling the residual, this coefficient explicitly controls how much cross-layer information is injected into each module, balancing accumulated-error correction against HA bias. In terms of coefficient estimation efficiency, we formulate residual-strength control as a closed-loop feedback problem, where module-level reconstruction error serves as the observable feedback signal for a Proportional-Integral-Derivative (PID)-based adaptive update, avoiding exhaustive coefficient search. Figure 1 provides a conceptual comparison among basic reconstruction, residual reconstruction, and MARR, together with representative performance improvements under 2-bit weight and 4-bit activation quantization (W2A4).

Our contributions are summarized as follows:

- We propose Module-Adaptive Residual Reconstruction (MARR), a new framework to address the suboptimal performance of residual reconstruction-based methods under low-bit PTQ.
- We introduce a module-specific scaling coefficient to balance the two-sided effect of residuals, and design a PID-based adaptive update strategy to efficiently estimate the coefficient using reconstruction-error feedback.
- Extensive experiments on LLMs and ViTs demonstrate that compared with residual reconstruction baselines, MARR achieves consistent improvements under low-bit quantization.

2 Related Work

Low-Bit Post-Training Quantization. Post-training quantization compresses pretrained models into low-precision representations without additional retraining, making it practical for large-scale models [22]. Existing PTQ methods improve low-bit robustness from several perspectives.



(a) Conceptual illustration of our proposed MARR (b) Perplexity comparison under W2A4 on WikiText2 [27]

Figure 1: Illustration of the motivation and effect of the proposed module-adaptive residual reconstruction. (a) Residual reconstruction improves GPTQ by adding global residual information; however, the residual term may also introduce the bias of Hessian-approximation (HA), leading to suboptimal quantization performance. Instead, MARR controls the residual contribution to balance the benefits and costs of residual. (b) Under W2A4 quantization, MARR consistently improves GPTAQ on WikiText2 [27], reducing perplexity by 9.82% on Llama3-8b and 14.94% on Llama2-7b.

Quantization-parameter-based methods [47] optimize clipping ranges, scaling factors, zero-points, or group-wise configurations to reduce discretization error. Redistribution-based methods reshape weight or activation distributions through smoothing [40], re-parameterization [20], rotation [3], or outlier migration [16], making models more amenable to low-bit quantization. Reconstruction-based methods [11, 19, 17] reduce quantization-induced reconstruction error through lightweight weight adjustment, while retaining efficiency via closed-form updates. In this work, we focus on reconstruction-based PTQ and investigate efficient residual reconstruction under low-bit quantization.

Reconstruction-Based PTQ. In the reconstruction-based PTQ, for a linear module we adopt the convention that an unadorned symbol denotes the FP quantity and its hat-marked counterpart denotes the quantized one. Specifically, w and $\hat{w}$ denote the FP weight vector and its Quant version, X and $\hat{X}$ denote the FP input (calibration reference) and the corresponding Quant input produced by already quantized preceding modules, and z and $\hat{z}$ denote the corresponding FP and quantized module outputs. Built upon the reconstruction principle of OBS [14] and OBQ [10], GPTQ [11] layer-wise preserves the original module output under the same quantized input $\hat{X}$. When quantizing the q -th coordinate, its reconstruction objective can be written as

$$\min_{\Delta w} \left\| (w + \Delta w) \hat{X} - w \hat{X} \right\|_F^2, \quad \text{s.t.} \quad \Delta w_q = \hat{w}_q - w_q. \quad (1)$$

Here, Δw denotes the weight perturbation introduced after quantizing the current coordinate, which adjusts the remaining weights to reconstruct the module output. Residual reconstruction further incorporates the mismatch between the FP and Quant inputs. A representative method is GPTAQ [19], which introduces the inter-layer activation residual $r = wX - w\hat{X}$ and formulates the reconstruction objective as

$$\min_{\Delta w} \left\| \Delta w \hat{X} - r \right\|_F^2, \quad \text{s.t.} \quad \Delta w_q = \hat{w}_q - w_q. \quad (2)$$

Solving Eq. (2) gives the closed-form update:

$$\Delta w = \frac{\hat{w}_q - w_q}{H_{qq}^{-1}} H_{q,:}^{-1} + r \hat{X}^\top H_{-q}^{-1}, \quad (3)$$

where $H = \hat{X} \hat{X}^\top$ is the input Hessian-approximation induced by the quantized input statistics, H^{-1} denotes its inverse, and $H_{-q}^{-1} = H^{-1} - \frac{H_{:,q}^{-1} H_{q,:}^{-1}}{H_{qq}^{-1}}$. The first term corresponds to the basic GPTQ reconstruction update, while the second term injects the propagated residual into the reconstruction process. These formulations show that residual reconstruction extends basic reconstruction by introducing cross-layer activation mismatch. However, while residual reconstruction accounts for cross-layer dependencies, the injected residual term may also bring additional residual-related bias arising from HA, motivating us to control its contribution in each module.

3 Proposed Method

3.1 The Two-Sided Effect of Residual Reconstruction

Hessian-Approximation. The local mean squared error (MSE) objective in reconstruction-based PTQ can be viewed as a tractable proxy of the Hessian-based local task-loss objective. A key step is to simplify the second-order derivative of the task-loss with respect to the module activation through HA [28]. Specifically, the HA can be expressed as

$$H_z = \nabla_z^2 \mathcal{L} \approx \text{diag}(c_1, \dots, c_d) = \bar{H}_z, \quad (4)$$

where H_z denotes the Hessian of the task-loss with respect to the output z [12], and $\bar{H}_z$ is its approximated counterpart under HA. $\nabla_z^2 \mathcal{L}$ denotes the second-order derivative of the task-loss with respect to z , and c_i is treated as i -th sample-independent constant. The HA in AdaRound [28] assumes the $\nabla_z^2 \mathcal{L}$ as a diagonal matrix whose entries are further treated as sample-independent constants, leading to a local MSE metric.

Residual Reconstruction under HA. Under low-bit quantization, the input of the current module is also perturbed by preceding quantized modules. Since the quantized output is $(w + \Delta w)\hat{X}$ while the FP reference is wX , the output perturbation is $\Delta z = (w + \Delta w)\hat{X} - wX = \Delta w\hat{X} - r$. Accordingly, the local residual reconstruction objective can be written as

$$\mathcal{J} = \mathbb{E} [\Delta z^\top H_z \Delta z] = \mathbb{E} [(\Delta w\hat{X} - r)^\top H_z (\Delta w\hat{X} - r)] \stackrel{\text{HA}}{\approx} \mathbb{E} [\|\Delta w\hat{X} - r\|_F^2]. \quad (5)$$

The detailed derivation is provided in Appendix A. The residual term r captures the FP-Quant mismatch propagated from preceding quantized modules, allowing the local reconstruction objective to account for cross-layer global dependency.

HA Bias in Residual. Although the residual term provides effective cross-layer correction, Eq. (5) still relies on the HA in Eq. (4). Let $\Delta H_z = H_z - \bar{H}_z$ denote the gap between the true task-loss Hessian and the simplified local MSE metric, so the HA bias of objective can be expanded and bounded as

$$\begin{aligned} \mathcal{E}_H(r) &= \left| \mathbb{E} [(\Delta w\hat{X} - r)^\top \Delta H_z (\Delta w\hat{X} - r)] \right| \\ &\leq \underbrace{\mathbb{E} [(\Delta w\hat{X})^\top \Delta H_z (\Delta w\hat{X})]}_{\text{weight-perturbation error}} + 2 \underbrace{\mathbb{E} [(\Delta w\hat{X})^\top \Delta H_z r]}_{\text{residual-related error}} + \mathbb{E} [r^\top \Delta H_z r]. \end{aligned} \quad (6)$$

The detailed derivation of Eq. (6) is provided in Appendix B. Eq. (6) shows that the HA bias contains two parts. The first term is induced by the weight perturbation $\Delta w\hat{X}$, while the latter two terms are introduced by the residual r . Therefore, while the residual term provides effective cross-layer correction, it also introduces additional HA biases under the Hessian-to-MSE surrogate.

Balancing the Two-Sided Effect of Residual. The above analysis shows that introducing the residual term has a two-sided effect. On the one hand, it helps reduce the accumulated quantization error ignored by basic reconstruction (Eq. (5)). On the other hand, it may introduce additional HA bias related to the residual r (Eq. (6)). Since both effects are governed by the strength of the residual term, scaling r provides a direct mechanism to adjust how much cross-layer correction is preserved and how much residual-related HA bias is introduced. To balance these two effects, we introduce a trade-off coefficient α and formulate the scaled-residual reconstruction objective as

$$\mathcal{J}_\alpha = \min_{\Delta w} \|\Delta w\hat{X} - \alpha r\|_F^2. \quad (7)$$

Here, α controls the strength of the residual in the local reconstruction objective. Following the same reconstruction procedure as GPTAQ, solving Eq. (7) with the coordinate quantization constraint $\Delta w_q = \hat{w}_q - w_q$ yields the following closed-form update:

$$\Delta w = \underbrace{\frac{\hat{w}_q - w_q}{H_{qq}^{-1}} H_{q,:}^{-1}}_{\text{basic GPTQ term}} + \underbrace{\alpha r \hat{X}^\top H_{-q}^{-1}}_{\text{scaled residual term}}. \quad (8)$$

The detailed derivation of Eq. (8) is provided in Appendix C. The first term is the basic GPTQ reconstruction update, while the second term is the residual reconstruction update controlled by α .

α	WikiText2 PPL
0.00	36.37
0.25	11.24
0.50	<u>9.89</u>
1.00	12.56
1.50	16.96
2.00	1.4e3
MARR	9.56

(a) Perplexity

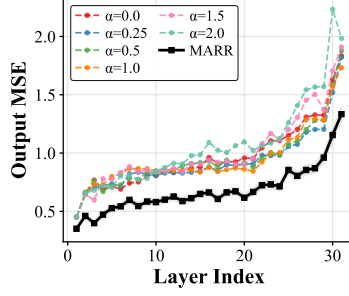
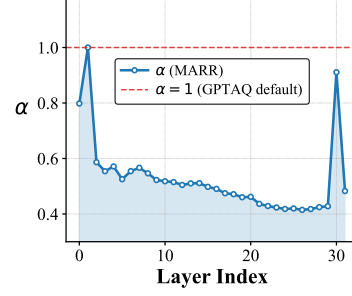
(b) Output MSE for several α values(c) α estimated by MARR per layer

Figure 2: Effect of the residual scaling coefficient α in GPTAQ under W2A4 on Llama2-7b. (a) The perplexity on WikiText2 [27] dataset for several fixed values of α together with MARR’s adaptive coefficient. (b) Output MSE of each layer for several fixed values of α . (c) Layer-wise α adaptively estimated by MARR for each layer, contrasted with the GPTAQ default $\alpha = 1$.

3.2 Module-Specific Residual Scaling

The scaled-residual update in Eq. (8) uses a single coefficient α to balance the cross-layer correction provided by the residual term and the residual-related HA bias (Eq. (6)). However, the balance point between these two effects is unlikely to be uniform across modules. Different modules carry different activation statistics, propagated residual magnitudes r , and HA gaps ΔH_z , so both the benefit (the magnitude of the cross-layer correction) and the cost (the residual-related error in $\mathcal{E}_H(r)$) depend on the module itself. This motivates us to assign a *module-specific* coefficient α_m , allowing each module to independently match its own balance point.

Empirical Evidence. To verify the effect of residual scaling, we sweep several fixed values of α on GPTAQ under W2A4. As shown in Figure 2, we report the final perplexity on WikiText2 [27], the layer-wise output MSE, and the layer-wise coefficient estimated by MARR. For clarity, instead of visualizing all modules, we report the output MSE of the last module in each layer as a representative case. Three observations can be obtained. *First*, the final perplexity is highly sensitive to the residual strength. A moderate α improves over both $\alpha = 0$ and the GPTAQ default $\alpha = 1$, while an overly large α sharply degrades perplexity, indicating that excessive residual strength can amplify residual-related HA bias. *Second*, MARR achieves lower layer-wise reconstruction MSE than fixed global coefficients in most layers, showing that adaptively controlling the residual strength can better balance accumulated-error correction and residual-related HA bias. *Third*, the coefficients estimated by MARR vary substantially across layers and often deviate from the GPTAQ default $\alpha = 1$. This directly shows that the desired residual strength is module-dependent; therefore, a single global α is insufficient, motivating our module-specific residual coefficient α_m .

Optimization Formulation. Building on this observation, we define the desired module-specific coefficient α_m^* as the one that minimizes the module-level output discrepancy:

$$\alpha_m^* = \arg \min_{\alpha_m} J_m(\alpha_m) = \arg \min_{\alpha_m} D(z_m, \hat{z}_m(\alpha_m)), \quad (9)$$

where z_m is the FP module output, $\hat{z}_m(\alpha_m)$ is the quantized module output reconstructed using the residual coefficient α_m , and $D(\cdot, \cdot)$ is instantiated as the output MSE. Since each module has its own loss landscape $J_m(\cdot)$ shaped by its own r and ΔH_z , the optimal α_m^* is naturally module-specific. Therefore, the desired coefficient should be estimated separately for each module.

3.3 PID-Based Adaptive Estimation

Estimating α_m^* . Actually, this is difficult to achieve the optimal coefficient directly in Eq. (9), since exhaustive search requires repeated reconstruction over a pre-defined α grid for each module, while an analytic solution is difficult due to the discrete and non-smooth nature of low-bit quantization. Inspired by the closed-loop control principle in iterative parameter updates [2, 5, 33, 25, 26], we formulate the estimation of α_m as a zero-order feedback problem. Specifically, α_m is treated as the adjustable variable, and the module-level reconstruction objective $J_m(\alpha_m)$ provides the observable

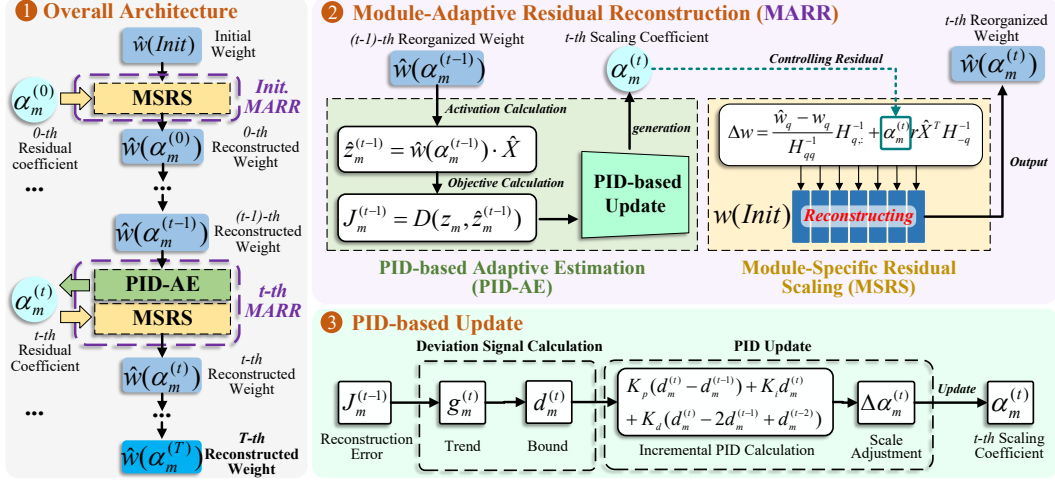


Figure 3: Overview of the PID-based adaptive update strategy for the module-specific residual scaling coefficient α_m .

feedback signal. This design avoids relying on gradients, since the reconstruction pipeline contains discrete quantization operations and repeated closed-form weight updates, making the response of $J_m(\alpha_m)$ to α_m piecewise non-smooth and difficult to differentiate reliably. As illustrated in Figure 3, the update is written as

$$\alpha_m^{(t)} = F_{\text{PID}}\left(J_m(\alpha_m^{(t-1)})\right), \quad t = 1, 2, \dots, T, \quad (10)$$

where $\alpha_m^{(t)}$ denotes the residual coefficient of module m at the t -th step, $F_{\text{PID}}(\cdot)$ is the PID-based update function, and $T = 3$ in our main experiments. We adopt a PID-based update because it provides a lightweight way to stabilize zero-order coefficient estimation from noisy reconstruction feedback: the proportional component reacts to the current reconstruction trend, the integral component accumulates persistent update direction, and the derivative component suppresses abrupt changes that may cause overshoot or oscillation. Accordingly, we instantiate $F_{\text{PID}}(\cdot)$ with three components: a trend-based deviation signal, an incremental PID update rule, and an early stopping criterion.

Deviation Signal of PID. Since low-bit PTQ generally leaves an inherent quantization error floor [48], the absolute value of $J_m(\alpha_m)$ is not a suitable zero-tracking target. We instead use the finite-difference trend of the reconstruction objective with respect to α_m as the feedback signal:

$$g_m^{(t)} = \frac{(J_m(\alpha_m^{(t-1)}) - J_m(\alpha_m^{(t-2)})) / (J_m(\alpha_m^{(0)}) + \varepsilon_J)}{\alpha_m^{(t-1)} - \alpha_m^{(t-2)} + \varepsilon_\alpha}, \quad d_m^{(t)} = \tanh(-\beta g_m^{(t)}), \quad (11)$$

where $\varepsilon_J = 10^{-8}$ normalizes the objective variation, $\varepsilon_\alpha = 10^{-6}$ stabilizes the finite-difference denominator, and $\beta = 10.0$ controls the response sensitivity. Here, $g_m^{(t)}$ measures whether the recent change of α_m reduces the reconstruction error, and $\tanh(\cdot)$ bounds the response for stable updates.

Update Rule of PID. Because $J_m(\alpha_m)$ is non-smooth under low-bit quantization, directly using the instantaneous trend may cause oscillation or overshoot. We therefore adopt the incremental PID rule to aggregate multi-step feedback:

$$\Delta\alpha_m^{(t)} = K_p(d_m^{(t)} - d_m^{(t-1)}) + K_i d_m^{(t)} + K_d(d_m^{(t)} - 2d_m^{(t-1)} + d_m^{(t-2)}), \quad (12)$$

$$\alpha_m^{(t)} = \alpha_m^{(t-1)} + \Delta\alpha_m^{(t)}, \quad (13)$$

where K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively, all empirically set to 1.0. This update smooths the search trajectory and stabilizes the estimation of the module-specific coefficient.

Table 1: Compared results on large language models. We report perplexity on WikiText2 and C4, together with zero-shot accuracy on six downstream tasks.

Model	W/A	Method	Wiki2 ↓	C4 ↓	PiQA	ARC-E	ARC-C	HellaSwag	WinoGrande	BoolQ	Avg ↑
Llama2-7b	W4A4	FP	5.47	6.90	79.00	74.60	46.50	76.00	68.90	77.70	70.50
		RTN	7.99	11.58	74.16	62.84	36.35	66.24	61.17	69.79	61.76
		GPTQ	6.01	8.16	77.69	71.76	42.49	73.17	65.04	75.32	67.58
		ResComp	5.88	8.04	77.86	70.46	42.69	73.72	65.23	73.06	67.17
		GPTAQ	5.86	7.98	76.50	70.37	41.98	73.52	65.90	73.64	66.98
		ResComp-MARR	5.84 (↓0.68%)	8.03 (↓0.12%)	76.88	69.91	42.24	73.70	66.06	74.71	67.25 (↑0.11%)
		GPTAQ-MARR	5.83 (↓0.51%)	7.92 (↓0.75%)	77.04	71.46	43.77	73.57	67.96	73.85	67.94 (↑1.43%)
	W2A4	RTN	7.7e3	9.0e3	50.76	25.59	28.58	26.38	48.54	37.83	36.28
		GPTQ	36.37	62.87	56.80	33.38	21.76	31.77	52.33	49.79	40.97
		ResComp	11.00	23.13	60.79	44.17	25.02	41.79	53.41	62.06	47.87
		GPTAQ	11.24	19.47	62.13	44.15	25.68	42.70	52.64	58.59	47.65
		ResComp-MARR	9.92 (↓9.82%)	21.85 (↓5.53%)	60.94	43.56	25.17	42.81	55.49	62.05	48.34 (↑0.98%)
		GPTAQ-MARR	9.56 (↓14.95%)	15.53 (↓20.24%)	61.48	43.90	25.26	43.84	57.38	61.31	48.86 (↑2.54%)
Llama2-13b	W4A4	FP	4.88	6.41	80.50	77.50	49.20	79.40	72.40	80.60	73.30
		RTN	5.93	8.34	77.37	74.03	44.80	73.73	68.82	76.09	69.14
		GPTQ	5.29	7.37	78.24	74.83	46.67	76.90	69.53	79.14	70.88
		ResComp	5.19	7.28	78.67	74.45	47.87	77.10	70.32	80.03	71.41
		GPTAQ	5.18	7.22	78.24	73.91	46.84	77.10	69.46	78.23	70.63
		ResComp-MARR	5.16 (↓0.58%)	7.26 (↓0.27%)	79.38	75.17	47.78	77.11	70.96	78.35	71.46 (↑0.07%)
		GPTAQ-MARR	5.14 (↓0.77%)	7.17 (↓0.69%)	78.73	73.99	46.84	77.27	70.48	77.46	70.79 (↑0.23%)
	W2A4	RTN	5.6e3	5.5e3	48.59	27.19	27.13	25.02	51.78	37.83	36.26
		GPTQ	13.55	30.81	60.94	41.29	24.49	41.35	53.35	62.05	47.24
		ResComp	8.31	17.90	63.76	51.76	26.55	47.65	54.22	60.29	50.70
		GPTAQ	8.62	15.61	67.03	54.83	31.08	48.54	56.31	62.01	53.30
		ResComp-MARR	7.97 (↓4.09%)	17.61 (↓1.62%)	64.74	48.27	28.75	48.67	57.62	62.91	51.83 (↑2.23%)
		GPTAQ-MARR	7.80 (↓9.51%)	15.18 (↓2.75%)	66.38	51.39	30.38	51.93	59.04	64.16	53.88 (↑1.09%)
Llama3-8b	W4A4	FP	6.44	9.61	80.70	77.70	53.70	79.10	73.20	81.10	74.30
		RTN	9.92	15.85	74.10	65.28	40.02	70.26	65.19	74.40	64.88
		GPTQ	7.77	12.54	75.73	72.18	44.28	73.81	66.69	76.15	68.14
		ResComp	7.42	12.39	78.02	72.22	47.44	74.72	67.64	76.79	69.47
		GPTAQ	7.36	12.22	78.24	71.80	46.76	74.94	67.88	77.86	69.58
		ResComp-MARR	7.36 (↓0.81%)	12.35 (↓0.32%)	78.18	74.62	46.16	74.97	67.40	76.70	69.67 (↑0.29%)
		GPTAQ-MARR	7.31 (↓0.68%)	12.15 (↓0.57%)	77.97	73.95	47.35	75.17	68.67	76.97	70.01 (↑0.62%)
	W2A4	RTN	6.1e4	5.0e4	51.09	25.63	26.62	26.87	50.20	43.15	37.26
		GPTQ	133.60	196.59	53.48	32.95	21.25	30.09	50.75	46.12	39.11
		ResComp	17.49	55.22	58.25	38.32	23.74	38.54	53.20	61.59	45.44
		GPTAQ	19.33	34.07	58.70	40.49	23.21	38.96	54.22	62.29	46.31
		ResComp-MARR	17.27 (↓1.26%)	52.83 (↓4.32%)	57.56	37.37	23.98	38.35	55.25	61.50	45.67 (↑0.51%)
		GPTAQ-MARR	17.43 (↓9.83%)	31.59 (↓7.28%)	59.79	39.35	25.26	40.58	54.14	58.90	46.34 (↑0.06%)

Stopping Criterion of PID. To avoid unnecessary reconstruction overhead, we stop the update when the relative change of the reconstruction objective becomes sufficiently small:

$$\left| \frac{J_m(\alpha_m^{(t)}) - J_m(\alpha_m^{(t-1)})}{J_m(\alpha_m^{(0)}) + \varepsilon_J} \right| < \tau, \quad (14)$$

where $\tau = 10^{-5}$ controls the stopping tolerance for the relative objective change. The current $\alpha_m^{(t)}$ is then used as the final residual scaling coefficient for module m .

With the above three components, PID-based update in MARR efficiently estimates a module-specific α_m that better balances the two-sided effect of the residual reconstruction term, leading to lower module-level reconstruction error in most modules, as shown in Figure 2. The complete update procedure is summarized in Appendix E, and a practical discussion on the stability of the PID-based update is provided in Appendix F.

4 Experiments

4.1 Settings

In this paper, all experiments are conducted under the same PyTorch framework on a single NVIDIA A6000 GPU. For a fair comparison, all methods are re-run with the same calibration data, quantization settings, hyperparameters, and evaluation protocols. For LLMs, we follow the evaluation protocol of GPTAQ [19], reporting perplexity on WikiText2 [27] and C4 [30] together with zero-shot accuracy on six downstream tasks. For ViTs, we follow the protocol of FIMA-Q [39] and report ImageNet [31] top-1 accuracy. In low-bit settings, all methods adopt the same rotation-based transformation [3] to mitigate activation outliers and avoid severe performance collapse. More detailed experimental settings are provided in Appendix D.

Table 2: Compared results of MARR under weight-only quantization on three Llama-family models. We report perplexity on WikiText2 and C4 together with the average zero-shot accuracy on six downstream tasks under W3A16 and W2A16.

W/A	Method	Llama2-7b			Llama2-13b			Llama3-8b		
		Wiki2 ↓	C4 ↓	Avg ↑	Wiki2 ↓	C4 ↓	Avg ↑	Wiki2 ↓	C4 ↓	Avg ↑
W3A16	FP	5.47	6.90	70.50	4.88	6.41	73.30	6.44	9.61	74.30
	RTN	146.54	114.65	37.32	48.90	54.60	43.64	39.60	50.90	47.72
	GPTQ	6.10	8.70	67.67	5.36	7.71	71.30	7.55	13.08	70.62
	ResComp	5.92	8.44	67.67	5.25	7.58	71.09	7.30	12.76	70.11
	GPTAQ	5.89	8.36	67.44	5.22	7.50	71.34	7.28	12.57	70.30
	ResComp-MARR GPTAQ-MARR	5.88 (↓0.68%) 5.83 (↓1.02%)	8.37 (↓0.83%) 8.26 (↓1.20%)	67.25 (↓0.62%) 67.83 (↑0.58%)	5.22 (↓0.57%) 5.18 (↓0.77%)	7.54 (↓0.53%) 7.45 (↓0.67%)	70.57 (↓0.73%) 71.19 (↓0.21%)	7.26 (↓0.55%) 7.18 (↓1.37%)	12.71 (↓0.39%) 12.57 (↓0.00%)	70.19 (↑0.11%) 69.30 (↓1.42%)
W2A16	RTN	9.8e3	1.2e4	36.30	5.2e3	5.0e3	35.22	4.1e4	3.0e4	36.11
	GPTQ	21.33	41.76	45.10	9.91	18.72	52.99	20.30	59.17	48.52
	ResComp	9.39	18.86	51.85	7.65	16.36	56.09	13.82	43.71	50.67
	GPTAQ	9.36	18.99	52.03	7.71	15.08	54.51	13.52	36.47	51.92
	ResComp-MARR GPTAQ-MARR	8.71 (↓7.24%) 8.59 (↓8.23%)	18.40 (↓2.44%) 16.86 (↓11.22%)	52.13 (↑0.54%) 52.52 (↑0.94%)	7.42 (↓3.01%) 7.08 (↓8.17%)	16.22 (↓0.86%) 13.64 (↓9.55%)	54.23 (↓3.32%) 56.59 (↑3.81%)	13.70 (↓0.87%) 13.28 (↓1.78%)	42.73 (↓2.24%) 35.90 (↓1.56%)	48.63 (↓4.03%) 50.09 (↓3.52%)

Table 3: Compared results on vision transformers. We report top-1 accuracy (%) on ImageNet under different quantization settings, where higher values indicate better performance.

Method	W4A4			W3A3			W2A4		
	DeiT-T	DeiT-S	DeiT-B	DeiT-T	DeiT-S	DeiT-B	DeiT-T	DeiT-S	DeiT-B
FP	72.71	79.85	81.80	72.71	79.85	81.80	72.71	79.85	81.80
PTQ4ViT	53.95	70.81	78.53	7.69	24.57	57.32	0.68	1.15	27.14
RepQ-ViT	55.31	70.13	78.37	0.72	6.43	18.11	0.17	0.22	3.01
AdaLog	62.58	64.81	76.20	30.05	55.82	71.90	1.76	11.90	46.25
GPTQ	64.19	74.77	78.79	29.26	43.41	67.35	30.30	54.43	67.20
ResComp	63.25	75.01	78.27	29.78	40.51	67.14	30.49	55.22	69.64
GPTAQ	65.16	76.35	79.62	32.32	56.41	68.82	33.58	59.17	70.79
ResComp-MARR GPTAQ-MARR	63.66 (↑0.65%) 65.39 (↑0.35%)	75.05 (↑0.05%) 76.81 (↑0.60%)	78.68 (↑0.52%) 80.11 (↑0.62%)	30.10 (↑1.07%) 32.45 (↑0.40%)	41.58 (↑2.64%) 59.01 (↑4.61%)	68.07 (↑1.39%) 69.22 (↑0.58%)	30.50 (↑0.03%) 33.82 (↑0.71%)	55.48 (↑0.47%) 59.92 (↑1.27%)	70.29 (↑0.93%) 71.24 (↑0.64%)

4.2 Weight-Activation Quantization Results on LLMs

To evaluate MARR on large language models, we compare it with round-to-nearest (RTN) [28], the basic reconstruction method GPTQ [11], and residual reconstruction methods GPTAQ [19] and ResComp [17]. We further apply MARR to GPTAQ and ResComp, denoted as GPTAQ-MARR and ResComp-MARR. Experiments are conducted on Llama2-7b [36], Llama2-13b [36] and Llama3-8b [1] under W4A4 and W2A4 settings. Table 1 reports perplexity on WikiText2 and C4, together with zero-shot accuracy on six downstream tasks [4, 8, 44, 32, 7].

As shown in Table 1, MARR achieves competitive or better results than most quantization methods, especially under the challenging W2A4 setting. Compared with RTN and basic GPTQ, residual reconstruction methods benefit from explicitly accounting for the accumulated activation mismatch introduced by preceding quantized modules. Building on this advantage, MARR further improves GPTAQ and ResComp by modulating the residual contribution at the module level, which preserves useful accumulated-error correction while reducing residual-related HA bias.

4.3 Weight-Only Quantization Results on LLMs

We further evaluate MARR under weight-only quantization, where the activations remain in 16-bit precision. Following the same protocol, we apply RTN [28], GPTQ [11], ResComp [17], GPTAQ [19], and our GPTAQ-MARR/ResComp-MARR to Llama2-7b, Llama2-13b and Llama3-8b under W3A16 and W2A16. Table 2 summarizes the WikiText2 and C4 perplexity together with the average zero-shot accuracy over six downstream tasks; the per-task zero-shot accuracy is provided in Appendix G.

As shown in Table 2, GPTAQ-MARR achieves the lowest WikiText2 and C4 perplexity in all six model-bit-width combinations, and the largest gains appear under the more aggressive W2A16 setting (e.g., 8.23%/11.22% perplexity reduction over GPTAQ on Llama2-7b). This confirms that controlling the residual contribution remains beneficial when only the weights are quantized.

4.4 Results on Vision Transformers

We further evaluate MARR on vision transformers under the standard ImageNet PTQ setting [39]. We compare it with representative ViT-oriented PTQ methods, including PTQ4ViT [43], RepQ-ViT [20], and AdaLog [38], as well as reconstruction-based methods GPTQ [11], GPTAQ [19], and ResComp [17]. Besides W4A4 and W2A4, we also include W3A3, since visual representations in

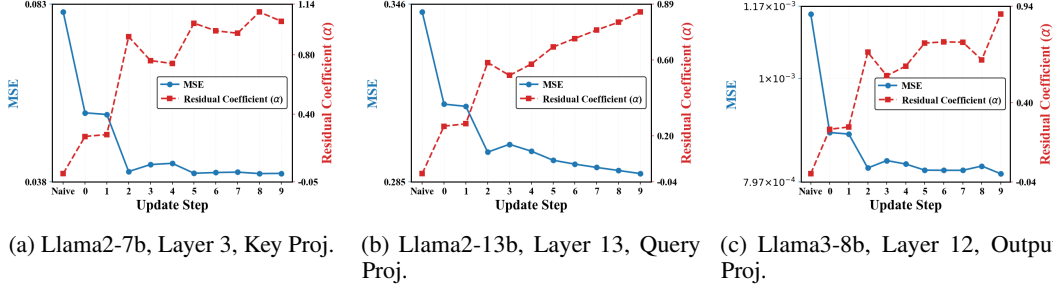


Figure 4: Trajectories of the residual scaling coefficient α_m and the module-level reconstruction MSE during the PID-based adaptive update on three representative modules from Llama2-7b, Llama2-13b, and Llama3-8b under W2A4.

ViTs often exhibit considerable redundancy, making sub-4-bit activation quantization a meaningful stress test for low-bit PTQ [39]. Table 3 reports the top-1 accuracy of DeiT-T, DeiT-S, and DeiT-B [35] under W4A4, W3A3, and W2A4.

As shown in Table 3, MARR achieves competitive or better performance than most baselines across different DeiT models and bit-widths. In particular, GPTAQ-MARR improves GPTAQ in most settings, indicating that the proposed module-specific residual scaling is also effective for vision transformers. This suggests that controlling the residual contribution can better balance accumulated-error correction and residual-related HA bias beyond LLMs, demonstrating the generality of MARR across Transformer architectures.

4.5 Ablation Study

We conduct ablation studies to analyze the key designs of MARR: module-specific residual scaling and PID-based adaptive update.

For module-specific residual scaling, we refer readers to Section 3.2, where fixed α sweeps in Figure 2 show that the preferred residual contribution varies across layers. This supports the necessity of using a module-specific coefficient α_m rather than a single global residual strength.

Second, we verify the effectiveness of the PID-based adaptive update. For visualization, we set the maximum update steps to 10 and record the evolution of α_m and module-level MSE, as shown in Figure 4. The results show that α_m progressively approaches a suitable residual strength, while the reconstruction error decreases and stabilizes within a few iterations. Since most modules already reach a low reconstruction error within $t = 3$, we set the maximum update step as $T = 3$ in the main experiments. These results confirm that the PID-based update can efficiently estimate a module-specific residual coefficient without expensive exhaustive search.

4.6 Limitations

We briefly outline three limitations of MARR. (i) The PID inner loop introduces a $2.1\text{--}2.7\times$ one-time PTQ overhead relative to GPTAQ. (ii) Under weight-only quantization, the gains on zero-shot accuracy become more limited or task-dependent, since the smaller FP-Quant activation mismatch weakens both the accumulated-error correction and the residual-related HA bias that MARR aims to balance. (iii) We fix $K_p = K_i = K_d = 1.0$ following prior PID-based deep-learning studies [2, 25], without a systematic gain sweep, leaving the sensitivity of these PID gains unexplored. More details and additional discussion are provided in Appendix I.

5 Conclusion

This paper studies residual reconstruction in low-bit PTQ from the perspective of residual contribution control. Although activation residuals can reduce accumulated cross-layer quantization errors, they may also introduce residual-related bias under the HA reconstruction objective. To address this issue, we propose MARR, a Module-Adaptive Residual Reconstruction framework that assigns a module-

specific scaling coefficient to the residual term. By explicitly controlling the residual contribution in each module, MARR balances accumulated-error correction and residual-induced approximation bias. We further design a PID-based adaptive update strategy that treats reconstruction error as feedback to efficiently estimate the coefficient without exhaustive search. Extensive experiments on LLMs and ViTs demonstrate that MARR improves residual reconstruction baselines under low-bit quantization, showing its effectiveness and generality across Transformer architectures.

References

- [1] AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- [2] Wangpeng An, Haoqian Wang, Qingyun Sun, Jun Xu, Qionghai Dai, and Lei Zhang. A pid controller approach for stochastic optimization of deep networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8522–8531, 2018.
- [3] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37:100213–100240, 2024.
- [4] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239.
- [5] Zhuotong Chen, Zihu Wang, Yifan Yang, Qianxiao Li, and Zheng Zhang. Pid control-based self-healing to improve the robustness of large language models. *arXiv preprint arXiv:2404.00828*, 2024.
- [6] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10558–10578, 2024.
- [7] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2924–2936, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300.
- [8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [10] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35:4475–4488, 2022.
- [11] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [12] Ruihao Gong, Xianglong Liu, Yuhang Li, Yunqiang Fan, Xiuying Wei, and Jinyang Guo. Pushing the limit of post-training quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [13] Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International journal of computer vision*, 129(6):1789–1819, 2021.
- [14] Babak Hassibi, David G Stork, and Gregory J Wolff. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*, pages 293–299. IEEE, 1993.
- [15] Zhi Jin, Yuwei Qiu, Kaihao Zhang, Hongdong Li, and Wenhan Luo. Mb-taylorformer v2: Improved multi-branch linear transformer expanded by taylor formula for image restoration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- [16] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. Owq: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 13355–13364, 2024.

- [17] Shuaiting Li, Juncan Deng, Kedong Xu, Rongtao Deng, Hong Gu, Minghan Jiang, Haibin Shen, and Kejie Huang. Rethinking residual errors in compensation-based llm quantization. *arXiv preprint arXiv:2604.07955*, 2026.
- [18] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. Brecq: Pushing the limit of post-training quantization by block reconstruction. *arXiv preprint arXiv:2102.05426*, 2021.
- [19] Yuhang Li, Ruokai Yin, Donghyun Lee, Shiting Xiao, and Priyadarshini Panda. Gptaq: Efficient finetuning-free quantization for asymmetric calibration. *arXiv preprint arXiv:2504.02692*, 2025.
- [20] Zhikai Li, Junrui Xiao, Lianwei Yang, and Qingyi Gu. Repq-vit: Scale reparameterization for post-training quantization of vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17227–17236, 2023.
- [21] Hou-I Liu, Marco Galindo, Hongxia Xie, Lai-Kuan Wong, Hong-Han Shuai, Yung-Hui Li, and Wen-Huang Cheng. Lightweight deep learning for resource-constrained environments: A survey. *ACM Computing Surveys*, 56(10):1–42, 2024.
- [22] Kai Liu, Qian Zheng, Kaiwen Tao, Zhiteng Li, Haotong Qin, Wenbo Li, Yong Guo, Xianglong Liu, Linghe Kong, Guihai Chen, et al. Low-bit model quantization for deep neural networks: A survey. *arXiv preprint arXiv:2505.05530*, 2025.
- [23] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
- [24] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinqant: Llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024.
- [25] Ruijun Ma, Bob Zhang, Yicong Zhou, Zhengming Li, and Fangyuan Lei. Pid controller-guided attention neural network learning for fast and effective real photographs denoising. *IEEE transactions on neural networks and learning systems*, 33(7):3010–3023, 2021.
- [26] Ruijun Ma, Shuyi Li, Bob Zhang, and Haifeng Hu. Meta pid attention network for flexible and efficient real-world noisy image denoising. *IEEE Transactions on Image Processing*, 31:2053–2066, 2022.
- [27] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
- [28] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International conference on machine learning*, pages 7197–7206. PMLR, 2020.
- [29] Yuwei Qiu, Kaihao Zhang, Chenxi Wang, Wenhan Luo, Hongdong Li, and Zhi Jin. Mb-taylorformer: Multi-branch efficient transformer expanded by taylor formula for image dehazing. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12802–12813, 2023.
- [30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- [31] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.
- [32] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021. doi: 10.1145/3474381.
- [33] Vipul Kumar Sharma, Wesley A Suttle, and S Sivaranjani. Globally optimal policy gradient algorithms for reinforcement learning with pid control policies. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [34] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. In *12th International Conference on Learning Representations, ICLR 2024*, 2024.

- [35] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021.
- [36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 10, 2023.
- [37] Xiuying Wei, Ruihao Gong, Yuhang Li, Xianglong Liu, and Fengwei Yu. Qdrop: Randomly dropping quantization for extremely low-bit post-training quantization. *arXiv preprint arXiv:2203.05740*, 2022.
- [38] Zhuguanyu Wu, Jiaxin Chen, Hanwen Zhong, Di Huang, and Yunhong Wang. Adalog: Post-training quantization for vision transformers with adaptive logarithm quantizer. In *European Conference on Computer Vision*, pages 411–427. Springer, 2024.
- [39] Zhuguanyu Wu, Shihe Wang, Jiayi Zhang, Jiaxin Chen, and Yunhong Wang. Fima-q: Post-training quantization for vision transformers by fisher information matrix approximation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14891–14900, 2025.
- [40] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [42] Liang Yang, Runjie Shi, Qiuliang Zhang, Zhen Wang, Xiaochun Cao, Chuan Wang, et al. Self-supervised graph neural networks via low-rank decomposition. *Advances in Neural Information Processing Systems*, 36:34295–34307, 2023.
- [43] Zhihang Yuan, Chenhao Xue, Yiqi Chen, Qiang Wu, and Guangyu Sun. Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization. In *European conference on computer vision*, pages 191–207. Springer, 2022.
- [44] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472.
- [45] Borui Zhao, Quan Cui, Renjie Song, Yiyu Qiu, and Jiajun Liang. Decoupled knowledge distillation. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 11953–11962, 2022.
- [46] Xingyu Zheng, Haotong Qin, Yuye Li, Haoran Chu, Jiakai Wang, Jinyang Guo, Michele Magno, and Xianglong Liu. First-order error matters: Accurate compensation for quantized large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 28883–28891, 2026.
- [47] Yunshan Zhong, Jiawei Hu, Mengzhao Chen, Rongrong Ji, et al. I&s-vit: An inclusive & stable method for pushing the limit of post-training vits quantization. *arXiv preprint arXiv:2311.10126*, 2023.
- [48] Yunshan Zhong, You Huang, Jiawei Hu, Yuxin Zhang, and Rongrong Ji. Towards accurate post-training quantization of vision transformers via error reduction. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(4):2676–2692, 2025.

Appendix

A Detailed Hessian-Approximation for Residual Reconstruction

We provide the detailed Hessian-approximation (HA) derivation of Eq. (5). Following the notation in Section 2, the residual-aware output perturbation is

$$\Delta z = (w + \Delta w)\hat{X} - wX = \Delta w\hat{X} - r, \quad r = wX - w\hat{X}. \quad (15)$$

Therefore, the Hessian-weighted residual reconstruction objective in Eq. (5) is

$$\mathcal{J} = \mathbb{E} \left[(\Delta w\hat{X} - r)^\top H_z (\Delta w\hat{X} - r) \right], \quad (16)$$

where $H_z = \nabla_z^2 \mathcal{L}$ is the Hessian of the task loss with respect to the module output.

We now expand the HA step following AdaRound [28]. For a single calibration sample, let $\hat{x}$ denote the quantized input and let r denote the corresponding residual target. For a linear module with weight matrix W , the perturbation induced by ΔW on the quantized path is $\Delta W\hat{x}$. Since the residual r is constructed from the FP-Quant input mismatch and is fixed when optimizing ΔW , the sensitivity of the i -th output perturbation to weight $W_{i,j}$ is

$$\frac{\partial(\Delta W_{i,:}\hat{x} - r_i)}{\partial W_{i,j}} = \hat{x}_j. \quad (17)$$

Thus, for two weights $W_{i,j}$ and $W_{m,o}$ in the same module, the corresponding Hessian element follows

$$\frac{\partial^2 \mathcal{L}}{\partial W_{i,j} \partial W_{m,o}} = \frac{\partial^2 \mathcal{L}}{\partial z_i \partial z_m} \hat{x}_j \hat{x}_o. \quad (18)$$

Writing Eq. (18) in matrix form gives

$$H(W) = \mathbb{E} [\hat{x}\hat{x}^\top \otimes H_z], \quad (19)$$

where $\otimes$ denotes the Kronecker product. This is the residual-aware counterpart of the Hessian decomposition used in AdaRound, with the input statistics computed from the quantized input $\hat{x}$.

The computational difficulty comes from H_z , which requires second-order information through the remaining network. Following the HA assumption, we approximate H_z as a diagonal matrix:

$$H_z = \nabla_z^2 \mathcal{L} \approx \text{diag} (\nabla_z^2 \mathcal{L}_{1,1}, \dots, \nabla_z^2 \mathcal{L}_{d,d}). \quad (20)$$

Substituting Eq. (20) into Eq. (16), the objective decomposes into independent row-wise subproblems:

$$\arg \min_{\Delta W_{k,:}} \mathbb{E} \left[\nabla_z^2 \mathcal{L}_{k,k} (\Delta W_{k,:}\hat{x} - r_k)^2 \right]. \quad (21)$$

Following AdaRound, we further assume that each diagonal entry is a sample-independent constant, i.e., $\nabla_z^2 \mathcal{L}_{k,k} = c_k$. This gives

$$\arg \min_{\Delta W_{k,:}} \mathbb{E} \left[c_k (\Delta W_{k,:}\hat{x} - r_k)^2 \right] \stackrel{(a)}{=} \arg \min_{\Delta W_{k,:}} \mathbb{E} \left[(\Delta W_{k,:}\hat{x} - r_k)^2 \right], \quad (22)$$

where (a) holds because c_k is a positive sample-independent constant and does not change the minimizer.

Equivalently, Eq. (22) can be expanded as

$$\begin{aligned} & \arg \min_{\Delta W_{k,:}} \mathbb{E} \left[(\Delta W_{k,:}\hat{x} - r_k)^2 \right] \\ &= \arg \min_{\Delta W_{k,:}} \Delta W_{k,:} \mathbb{E} [\hat{x}\hat{x}^\top] \Delta W_{k,:}^\top - 2\Delta W_{k,:} \mathbb{E} [\hat{x}r_k] + \mathbb{E} [r_k^2]. \end{aligned} \quad (23)$$

Compared with the basic AdaRound local MSE objective, the residual-aware objective introduces the cross term $\mathbb{E}[\hat{x}r_k]$, which pulls the reconstruction toward the FP-flow reference rather than only minimizing the perturbation $\Delta W_{k,:}\hat{x}$.

Stacking all calibration samples into the matrix $\hat{X}$ and the residual vector r_k , Eq. (22) becomes

$$\arg \min_{\Delta W_{k,:}} \left\| \Delta W_{k,:} \hat{X} - r_k \right\|_F^2. \quad (24)$$

For a row-wise weight vector w , this yields the residual-aware MSE surrogate:

$$\mathbb{E} \left[(\Delta w \hat{X} - r)^\top H_z (\Delta w \hat{X} - r) \right] \stackrel{\text{HA}}{\approx} \mathbb{E} \left[\left\| \Delta w \hat{X} - r \right\|_F^2 \right]. \quad (25)$$

Thus, the HA step in Eq. (5) follows the same row-wise Hessian-to-MSE reduction as AdaRound. The key difference is that the basic perturbation $\Delta w \hat{X}$ is replaced by the residual-aware perturbation $\Delta w \hat{X} - r$, so the resulting local MSE objective explicitly reconstructs the quantized-path output toward the FP-flow reference.

B Derivation of the Hessian-Approximation Error in Residual Reconstruction

We provide the detailed derivation of Eq. (6). Following the notation in the main text, the residual-aware output perturbation is

$$\Delta z = (w + \Delta w) \hat{X} - wX = \Delta w \hat{X} - r, \quad r = wX - w\hat{X}. \quad (26)$$

Let H_z denote the true task-loss Hessian with respect to the module output, and let $\bar{H}_z$ denote its HA counterpart used by the local reconstruction surrogate. The true Hessian-weighted residual objective is

$$\mathcal{J}_{\text{true}}(r) = \mathbb{E} \left[(\Delta w \hat{X} - r)^\top H_z (\Delta w \hat{X} - r) \right], \quad (27)$$

while the HA objective is

$$\mathcal{J}_{\text{HA}}(r) = \mathbb{E} \left[(\Delta w \hat{X} - r)^\top \bar{H}_z (\Delta w \hat{X} - r) \right]. \quad (28)$$

Therefore, the objective error introduced by HA is

$$\begin{aligned} \mathcal{E}_{\text{H}}(r) &= |\mathcal{J}_{\text{true}}(r) - \mathcal{J}_{\text{HA}}(r)| \\ &= \left| \mathbb{E} \left[(\Delta w \hat{X} - r)^\top (H_z - \bar{H}_z) (\Delta w \hat{X} - r) \right] \right|. \end{aligned} \quad (29)$$

By defining

$$\Delta H_z = H_z - \bar{H}_z, \quad (30)$$

Eq. (29) becomes

$$\mathcal{E}_{\text{H}}(r) = \left| \mathbb{E} \left[(\Delta w \hat{X} - r)^\top \Delta H_z (\Delta w \hat{X} - r) \right] \right|. \quad (31)$$

We now expand the quadratic form in Eq. (31). Let

$$a = \Delta w \hat{X} \quad (32)$$

denote the weight-induced output perturbation. Then

$$(a - r)^\top \Delta H_z (a - r) = a^\top \Delta H_z a - a^\top \Delta H_z r - r^\top \Delta H_z a + r^\top \Delta H_z r. \quad (33)$$

Since both H_z and $\bar{H}_z$ are symmetric curvature matrices, their difference ΔH_z is also symmetric. Hence,

$$a^\top \Delta H_z r = r^\top \Delta H_z a. \quad (34)$$

Substituting Eq. (34) into Eq. (33), we obtain

$$(a - r)^\top \Delta H_z (a - r) = a^\top \Delta H_z a - 2a^\top \Delta H_z r + r^\top \Delta H_z r. \quad (35)$$

Plugging Eq. (35) into Eq. (31) gives

$$\mathcal{E}_{\text{H}}(r) = \left| \mathbb{E} \left[a^\top \Delta H_z a - 2a^\top \Delta H_z r + r^\top \Delta H_z r \right] \right|. \quad (36)$$

Applying the triangle inequality and using $|\mathbb{E}[u]| \leq \mathbb{E}[|u|]$, we have

$$\mathcal{E}_H(r) \leq \mathbb{E}[|a^\top \Delta H_z a|] + 2\mathbb{E}[|a^\top \Delta H_z r|] + \mathbb{E}[|r^\top \Delta H_z r|]. \quad (37)$$

Finally, substituting $a = \Delta w \hat{X}$ back into Eq. (37), we obtain

$$\begin{aligned} \mathcal{E}_H(r) \leq & \underbrace{\mathbb{E}\left[\left|(\Delta w \hat{X})^\top \Delta H_z (\Delta w \hat{X})\right|\right]}_{\text{weight-perturbation error}} \\ & + \underbrace{2\mathbb{E}\left[\left|(\Delta w \hat{X})^\top \Delta H_z r\right|\right]}_{\text{residual-related error}} + \mathbb{E}[|r^\top \Delta H_z r|]. \end{aligned} \quad (38)$$

Eq. (38) corresponds to Eq. (6) in the main text. The first term is the Hessian-approximation bias caused by the weight-induced perturbation $\Delta w \hat{X}$, which also exists in basic reconstruction. The latter two terms are introduced by the residual r : the cross term $(\Delta w \hat{X})^\top \Delta H_z r$ captures the interaction between the weight perturbation and the propagated residual, while $r^\top \Delta H_z r$ measures the residual's own curvature mismatch under HA. Therefore, residual reconstruction not only provides cross-layer correction, but also brings additional residual-related Hessian-approximation bias under the local reconstruction surrogate.

C Derivation of the Residual-Scaled Closed-Form Update

We derive the closed-form solution of the scaled residual reconstruction objective in Eq. (7). Following the notation in the main text, X and $\hat{X}$ denote the FP input and the quantized input, respectively, and the propagated activation residual is defined as

$$r = wX - w\hat{X} = w(X - \hat{X}). \quad (39)$$

For a row-wise weight vector w , the scaled residual reconstruction problem is

$$\min_{\Delta w} \left\| \Delta w \hat{X} - \alpha r \right\|_F^2, \quad \text{s.t.} \quad \Delta w_q = \hat{w}_q - w_q. \quad (40)$$

Here, α controls the strength of the residual term, and the coordinate constraint enforces that the q -th weight is quantized to $\hat{w}_q$.

Let

$$d_q = \hat{w}_q - w_q, \quad H = \hat{X} \hat{X}^\top. \quad (41)$$

We use $e_q \in \mathbb{R}^{1 \times n}$ to denote the row basis vector whose q -th entry is one and all other entries are zero. Then the coordinate constraint can be written as

$$\Delta w e_q^\top = d_q. \quad (42)$$

The Lagrangian of Eq. (40) is

$$\mathcal{L}(\Delta w, \lambda) = \left\| \Delta w \hat{X} - \alpha r \right\|_F^2 + \lambda (\Delta w e_q^\top - d_q), \quad (43)$$

where λ is the Lagrange multiplier. Expanding the derivative with respect to Δw gives

$$\frac{\partial}{\partial \Delta w} \left\| \Delta w \hat{X} - \alpha r \right\|_F^2 = 2(\Delta w \hat{X} - \alpha r) \hat{X}^\top = 2\Delta w H - 2\alpha r \hat{X}^\top. \quad (44)$$

Therefore, the KKT conditions are

$$\begin{cases} 2\Delta w H - 2\alpha r \hat{X}^\top + \lambda e_q = 0, \\ \Delta w e_q^\top = d_q. \end{cases} \quad (45)$$

From the first equation of Eq. (45), we obtain

$$\Delta w H = \alpha r \hat{X}^\top - \frac{\lambda}{2} e_q. \quad (46)$$

Multiplying both sides by H^{-1} from the right yields

$$\Delta w = \alpha r \hat{X}^\top H^{-1} - \frac{\lambda}{2} e_q H^{-1}. \quad (47)$$

Since $e_q H^{-1} = H_{q,:}^{-1}$, Eq. (47) becomes

$$\Delta w = \alpha r \hat{X}^\top H^{-1} - \frac{\lambda}{2} H_{q,:}^{-1}. \quad (48)$$

We next use the coordinate constraint to solve for λ . Taking the q -th coordinate of Eq. (48), we have

$$\Delta w_q = \alpha r \hat{X}^\top H_{:,q}^{-1} - \frac{\lambda}{2} H_{qq}^{-1}. \quad (49)$$

Using $\Delta w_q = d_q$, we obtain

$$d_q = \alpha r \hat{X}^\top H_{:,q}^{-1} - \frac{\lambda}{2} H_{qq}^{-1}, \quad (50)$$

and thus

$$\frac{\lambda}{2} = \frac{\alpha r \hat{X}^\top H_{:,q}^{-1} - d_q}{H_{qq}^{-1}}. \quad (51)$$

Substituting Eq. (51) into Eq. (48), we get

$$\begin{aligned} \Delta w &= \alpha r \hat{X}^\top H^{-1} - \frac{\alpha r \hat{X}^\top H_{:,q}^{-1} - d_q}{H_{qq}^{-1}} H_{q,:}^{-1} \\ &= \frac{d_q}{H_{qq}^{-1}} H_{q,:}^{-1} + \alpha r \hat{X}^\top \left(H^{-1} - \frac{H_{:,q}^{-1} H_{q,:}^{-1}}{H_{qq}^{-1}} \right). \end{aligned} \quad (52)$$

Following GPTAQ, we define the inverse Hessian after eliminating the q -th coordinate as

$$H_{-q}^{-1} = H^{-1} - \frac{H_{:,q}^{-1} H_{q,:}^{-1}}{H_{qq}^{-1}}. \quad (53)$$

Then Eq. (52) becomes

$$\Delta w = \frac{d_q}{H_{qq}^{-1}} H_{q,:}^{-1} + \alpha r \hat{X}^\top H_{-q}^{-1}. \quad (54)$$

Finally, substituting $d_q = \hat{w}_q - w_q$ gives

$$\Delta w = \frac{\hat{w}_q - w_q}{H_{qq}^{-1}} H_{q,:}^{-1} + \alpha r \hat{X}^\top H_{-q}^{-1}. \quad (55)$$

Eq. (55) is the closed-form update used in Eq. (8). The first term is the basic GPTQ reconstruction update induced by the coordinate quantization constraint, while the second term is the residual reconstruction update scaled by α . When $\alpha = 0$, the update reduces to the basic GPTQ-style reconstruction; when $\alpha = 1$, it recovers the standard GPTAQ-style residual reconstruction update. Therefore, α explicitly controls how strongly the propagated residual is injected into the closed-form reconstruction update.

D Experimental Settings

We provide detailed experimental settings for both LLM and ViT experiments below.

Hardware and software. All experiments are conducted on a single NVIDIA RTX A6000 GPU (48 GB) on a server equipped with two Intel Xeon Platinum 8352V CPUs (2×36 physical cores @ 2.10 GHz, 144 logical threads in total) and 512 GB of DDR4 system memory, running Ubuntu 20.04 LTS (Linux kernel 5.15) with NVIDIA driver 550.67. The PTQ pipeline is implemented in Python 3.10 and PyTorch 2.1 with CUDA 12.1. Each run uses identical random seeds across compared methods to ensure reproducibility, and reported numbers correspond to a single PTQ pass per setting.

Rotation-based pre-processing. For all compared methods in low-bit settings, we apply the same QuaRot [3] rotation-based transformation to mitigate activation outliers and avoid severe performance collapse. The rotation is applied identically across baselines (RTN, GPTQ, GPTAQ, ResComp) and our method (GPTAQ-MARR, ResComp-MARR) to ensure a fair comparison.

LLM settings. We follow the evaluation protocol of GPTAQ [19]. The calibration set consists of 128 samples randomly drawn from WikiText2 [27], each with a sequence length of 2048. We report perplexity on WikiText2 [27] and C4 [30] with sequence length 2048, and zero-shot accuracy on six downstream tasks: PiQA [4], ARC-E and ARC-C [8], HellaSwag [44], WinoGrande [32], and BoolQ [7], evaluated using the standard lm-evaluation-harness pipeline. Models in the main experiments include Llama2-7b, Llama2-13b, and Llama3-8b; bit-width settings are W4A4 and W2A4. Weights are quantized at uniform precision per output channel, and activations are quantized per-token in dynamic mode.

ViT settings. We follow the evaluation protocol of FIMA-Q [39]. The calibration set consists of 1024 unlabeled images randomly sampled from the ImageNet [31] training set. We report top-1 accuracy on the ImageNet validation set (50K images). Models in the main experiments include DeiT-T, DeiT-S, and DeiT-B; bit-width settings are W4A4, W3A3, and W2A4. The same QuaRot rotation as for LLMs is applied to all compared ViT methods in low-bit settings to mitigate activation outliers.

MARR hyperparameters. The maximum number of update steps is set to $T = 3$. The PID-style weighting coefficients are uniformly set to $K_p = K_i = K_d = 1.0$. The response sensitivity is $\beta = 10.0$, the stabilizing constants are $\varepsilon_J = 10^{-8}$ and $\varepsilon_\alpha = 10^{-6}$, and the stopping threshold is $\tau = 10^{-5}$. The initial residual scaling coefficient is $\alpha_m^{(0)} = 1.0$ for all modules. These hyperparameters are kept fixed across all LLM and ViT experiments without per-task tuning.

E PID-Based Adaptive Update Algorithm

The complete procedure of the PID-based adaptive update is summarized in Table 4. For a given module m , the algorithm estimates the module-specific residual scaling coefficient α_m using the module-level reconstruction objective $J_m(\alpha_m)$ as feedback.

Table 4: Procedure of the PID-based adaptive update for estimating the module-specific residual scaling coefficient α_m in MARR.

Step	Operation
Input	Module m with FP output z_m , quantized input $\hat{X}_m$, and residual term r_m ; reconstruction objective $J_m(\alpha) = D(z_m, \hat{z}_m(\alpha))$; hyperparameters $T, K_p, K_i, K_d, \beta, \tau, \varepsilon_J$, and ε_α .
Output	Final module-specific residual scaling coefficient α_m^{final} .
1	Initialize two starting coefficients $\alpha_m^{(-1)} \leftarrow 0$ and $\alpha_m^{(0)} \leftarrow 1$; initialize $d_m^{(-1)} \leftarrow 0$ and $d_m^{(0)} \leftarrow 0$.
2	For $s \in \{-1, 0\}$, reconstruct module m using the scaled-residual closed-form update in Eq. (8) with $\alpha = \alpha_m^{(s)}$, and compute $J_m(\alpha_m^{(s)})$.
3	for $t = 1, 2, \dots, T$ do
4	Compute the finite-difference trend $g_m^{(t)}$ and bounded deviation signal $d_m^{(t)}$ according to Eq. (11).
5	Compute the PID increment $\Delta\alpha_m^{(t)}$ according to Eq. (12).
6	Update $\alpha_m^{(t)}$ according to Eq. (13).
7	Reconstruct module m using Eq. (8) with $\alpha = \alpha_m^{(t)}$, and compute $J_m(\alpha_m^{(t)})$.
8	if the stopping criterion in Eq. (14) is satisfied then break .
9	end for
10	return $\alpha_m^{\text{final}} \leftarrow \alpha_m^{(t)}$.

In the main experiments, we set $T = 3$. The two initial coefficients $\alpha_m^{(-1)} = 0$ and $\alpha_m^{(0)} = 1$ correspond to removing the residual term and using the original residual strength, respectively, which provides the first finite-difference trend for the PID update.

F Practical Stability Discussion of PID-Based Update

The PID-based update in MARR is designed as a lightweight feedback estimator for the module-specific residual coefficient, rather than as a globally convergent optimizer for the exact optimum of $J_m(\alpha_m)$. This is because the reconstruction objective involves discrete low-bit quantization and closed-form weight reconstruction, making $J_m(\alpha_m)$ generally non-smooth and difficult to optimize with a standard gradient-based solver. Therefore, instead of claiming a strict global convergence guarantee, we discuss why the proposed PID update is stable in practice and why it can reach a reliable coefficient estimate within a few steps.

Bounded feedback signal. The feedback signal is computed from the finite-difference trend of the module-level reconstruction objective, and then passed through a $\tanh(\cdot)$ function:

$$d_m^{(t)} = \tanh(-\beta g_m^{(t)}). \quad (56)$$

This design bounds the response within $[-1, 1]$, preventing a noisy or abrupt change in $J_m(\alpha_m)$ from producing an excessively large update. This is particularly important for low-bit quantization, where the reconstruction error may change non-smoothly when the quantized weights switch between discrete values.

Negative-feedback behavior. The sign of $d_m^{(t)}$ naturally forms a negative-feedback mechanism. If the recent change of α_m increases the reconstruction error, the finite-difference trend $g_m^{(t)}$ becomes positive, and $\tanh(-\beta g_m^{(t)})$ produces a negative response, which pushes α_m back toward a smaller residual contribution. Conversely, if increasing α_m decreases the reconstruction error, the response becomes positive and encourages a larger residual contribution. In this way, the update direction is determined by whether the previous coefficient adjustment improves or worsens the reconstruction objective.

Oscillation suppression by the PID form. Directly updating α_m using only the instantaneous trend may be sensitive to local fluctuations of $J_m(\alpha_m)$. The incremental PID rule alleviates this issue by combining three types of feedback. The proportional term reacts to the current change of the deviation signal, the integral term accumulates persistent improvement or degradation trends, and the derivative term captures the second-order change of the deviation signal. As a result, the update does not rely solely on a single noisy measurement of $J_m(\alpha_m)$, but instead uses the recent trajectory of reconstruction errors to adjust the coefficient more smoothly.

Finite-step estimation instead of long-horizon optimization. MARR only uses a small number of PID update steps, with $T = 3$ in our main experiments. Therefore, the PID update is not intended to perform long-horizon iterative optimization. Instead, it serves as a finite-step estimator that quickly adjusts the initial coefficient $\alpha_m = 1$ toward a better module-specific residual strength. This design keeps the additional reconstruction overhead low and avoids the risk of persistent oscillation that may appear in long iterative procedures.

Early stopping. We further stop the update when the relative change of the reconstruction objective becomes sufficiently small:

$$\left| \frac{J_m(\alpha_m^{(t)}) - J_m(\alpha_m^{(t-1)})}{J_m(\alpha_m^{(0)}) + \varepsilon_J} \right| < \tau. \quad (57)$$

This criterion prevents unnecessary coefficient updates once the reconstruction objective has become stable. Therefore, even if small fluctuations exist in the local objective landscape, the update is terminated when further changes provide negligible improvement.

Empirical stabilization. The above designs make the PID update stable in practice: the response is bounded, the direction follows a negative-feedback principle, the PID form smooths the update trajectory, and early stopping avoids unnecessary oscillation. As shown in our empirical results, the coefficient and reconstruction error typically stabilize within three update steps. This indicates that the proposed PID strategy is sufficient for estimating an effective module-specific residual coefficient without expensive grid search.

Table 5: Per-task zero-shot accuracy of MARR under weight-only quantization on three Llama-family models. We report top-1 accuracy on six downstream tasks together with their average under W3A16 and W2A16. Higher is better.

Model	W	Method	PiQA	ARC-E	ARC-C	HellaSwag	WinoGrande	BoolQ	Avg $\uparrow$
Llama2-7b	W3A16	FP	79.00	74.60	46.50	76.00	68.90	77.70	70.50
		RTN	53.20	30.60	24.90	29.10	48.30	37.80	37.32
		GPTQ	77.10	71.60	43.30	71.10	67.00	75.90	67.67
		ResComp	76.70	72.00	42.10	71.90	68.20	75.20	67.67
		GPTAQ	77.40	71.80	42.40	72.00	66.90	74.10	67.44
		ResComp-MARR	77.30	72.50	42.70	71.50	66.50	73.10	67.25 ($\downarrow 0.62\%$)
		GPTAQ-MARR	77.40	71.50	42.50	72.40	68.80	74.60	67.83 ($\uparrow 0.58\%$)
	W2A16	RTN	50.50	25.60	28.20	26.50	49.20	37.80	36.30
		GPTQ	58.70	38.80	23.40	35.40	52.80	61.60	45.10
		ResComp	64.70	49.90	28.50	46.60	58.30	63.20	51.85
		GPTAQ	64.80	49.80	27.60	46.80	59.20	64.10	52.03
		ResComp-MARR	65.70	51.40	25.90	47.50	60.20	62.20	52.13 ($\uparrow 0.54\%$)
		GPTAQ-MARR	65.70	49.60	27.70	47.80	60.50	63.90	52.52 ($\uparrow 0.94\%$)
Llama2-13b	W3A16	FP	80.50	77.50	49.20	79.40	72.40	80.60	73.30
		RTN	57.20	33.60	23.80	32.10	53.00	62.10	43.64
		GPTQ	78.30	75.80	47.10	76.00	70.80	79.80	71.30
		ResComp	78.40	76.20	47.20	76.40	71.80	76.50	71.09
		GPTAQ	78.80	75.70	47.40	76.20	71.70	78.30	71.34
		ResComp-MARR	78.10	74.90	46.70	76.00	71.00	76.70	70.57 ($\downarrow 0.73\%$)
		GPTAQ-MARR	78.70	75.10	46.90	76.30	71.70	78.50	71.19 ($\downarrow 0.21\%$)
	W2A16	RTN	48.40	26.40	26.60	25.10	47.00	37.80	35.22
		GPTQ	66.50	51.40	28.30	51.20	58.20	62.40	52.99
		ResComp	69.00	56.40	32.30	53.90	61.20	63.70	56.09
		GPTAQ	67.80	53.20	31.00	51.80	60.90	62.40	54.51
		ResComp-MARR	67.40	53.20	30.80	50.50	61.10	62.40	54.23 ($\downarrow 3.32\%$)
		GPTAQ-MARR	69.00	55.80	32.90	54.50	63.40	63.90	56.59 ($\uparrow 3.81\%$)
Llama3-8b	W3A16	FP	80.70	77.70	53.70	79.10	73.20	81.10	74.30
		RTN	64.90	40.40	24.20	45.30	60.90	50.60	47.72
		GPTQ	77.60	73.50	46.40	73.80	71.70	80.80	70.62
		ResComp	77.60	72.40	46.40	73.70	72.00	78.70	70.11
		GPTAQ	78.10	73.40	46.70	73.80	71.90	77.80	70.30
		ResComp-MARR	77.20	73.70	46.40	73.20	70.60	80.10	70.19 ($\uparrow 0.03\%$)
		GPTAQ-MARR	78.20	72.50	46.70	73.70	70.90	73.90	69.30 ($\downarrow 1.42\%$)
	W2A16	RTN	51.60	25.40	25.10	26.80	50.00	37.80	36.11
		GPTQ	60.50	43.00	25.00	41.90	57.00	63.80	48.52
		ResComp	63.00	46.40	26.80	46.10	59.40	62.20	50.67
		GPTAQ	65.20	48.00	27.80	47.30	58.90	64.30	51.92
		ResComp-MARR	60.50	41.60	25.70	43.50	57.00	63.50	48.63 ($\downarrow 4.03\%$)
		GPTAQ-MARR	61.20	42.20	28.60	45.90	56.50	66.10	50.09 ($\downarrow 3.52\%$)

G Other Weight-Only Quantization Experiments

Table 2 in the main paper has summarized weight-only quantization with WikiText2 and C4 perplexity together with the average zero-shot accuracy. Here, we further provide the per-task zero-shot accuracy on six downstream tasks (PiQA, ARC-E, ARC-C, HellaSwag, WinoGrande, BoolQ) for the same set of models and bit-widths in Table 5.

Across the three Llama-family models, the per-task accuracy patterns are consistent with the perplexity gains in Table 2: GPTAQ-MARR remains competitive with or surpasses its GPTAQ backbone on Llama2-7b and Llama2-13b at W2A16, while showing slight zero-shot regressions on Llama3-8b. This task-level mismatch is the same effect already discussed in Appendix I: the residual coefficient α_m is tuned to minimize module-level reconstruction error on a small calibration set, which transfers tightly to perplexity but only loosely to certain downstream tasks (e.g., BoolQ) whose label distributions differ from the calibration corpus.

H Quantization Efficiency

To quantify the additional one-time PTQ cost introduced by MARR’s PID-based update, we measure the wall-clock time of a single quantization pass for each compared method on three Llama-family models under the W2A16 setting (matching the most aggressive bit-width in Table 5). All measurements are performed on a single NVIDIA A6000 GPU with the same calibration set (128 samples of length 2048) and the same Hadamard rotation pre-processing as in our main experiments. Since the GPTQ-style reconstruction passes only operate on weight matrices during quantization, the relative

cost across methods is essentially independent of the activation bit-width. Results are reported in Table 6.

Table 6: Wall-clock time of a single PTQ pass on Llama-family models under W2A16. Time is reported in minutes; relative cost is shown with respect to GPTAQ, which is the residual-reconstruction backbone of GPTAQ-MARR. RTN involves no GPTQ-style iterative update and finishes in well under one minute.

Method	Llama2-7b		Llama2-13b		Llama3-8b	
	Time (min)	Relative	Time (min)	Relative	Time (min)	Relative
RTN	<1	<0.1×	<1	<0.1×	<1	<0.1×
GPTQ	13.7	0.5×	23.1	0.5×	15.1	0.4×
ResComp	29.9	1.0×	47.6	0.9×	34.6	0.8×
GPTAQ	30.0	1.0×	50.1	1.0×	41.1	1.0×
ResComp-MARR	74.3	2.5×	132.8	2.7×	107.4	2.6×
GPTAQ-MARR	72.7	2.4×	125.1	2.5×	88.1	2.1×

Two trends are visible in Table 6. First, GPTQ runs at only 0.4–0.5× of GPTAQ’s cost since it skips the extra forward propagation through the already-quantized prefix that GPTAQ requires to compute the inter-layer residual term in Eq. (2); ResComp shares this propagation pattern and therefore lands within 0.8–1.0× of GPTAQ. Second, MARR variants add a 2.1–2.7× multiplier on top of GPTAQ, regardless of whether GPTAQ or ResComp serves as the underlying residual-reconstruction backbone. This overhead originates entirely from the inner feedback loop: for each module we run $T = 3$ additional update steps to evaluate and refine the residual scaling coefficient α_m , and the dominant cost per step is re-quantizing the candidate weights and re-evaluating the resulting reconstruction error. The total cost therefore scales approximately linearly with T and is independent of the target bit-width.

In absolute terms, even MARR’s slowest pass on Llama2-13b finishes within about 2.2 hours on a single A6000, which we view as an acceptable one-time PTQ overhead given the consistent perplexity and zero-shot accuracy gains reported in Table 1 and Table 5. Further speedups are possible by lowering T , since the trajectories in Appendix J show that most modules already reach a near-optimal α_m within one or two updates, or by parallelizing the feedback iterations across calibration sub-batches, which we leave to future work.

I Limitations

We acknowledge several limitations of MARR.

Additional Quantization Overhead. MARR estimates a module-specific residual coefficient through several additional reconstruction passes, leading to a 2.1–2.7× overhead over GPTAQ (Appendix H). However, this overhead should be interpreted within the context of reconstruction-based PTQ, which is inherently lightweight because it relies on closed-form weight updates rather than back-propagation or retraining. Therefore, although MARR is slower than its residual reconstruction baseline, it still remains much more efficient than back-propagation-based reconstruction methods such as BRECQ [18] and is practical for post-training quantization.

Limited Zero-Shot Gains under Weight-Only Quantization. Under weight-only quantization, activations remain in high precision, so the mismatch between the FP and Quant activation flows is much weaker than in weight-activation quantization. Consequently, the residual term r becomes smaller, and the residual-related HA bias analyzed in Eq. (6) is also less pronounced. In this case, the correction–bias trade-off controlled by MARR still exists, but its effect size becomes smaller because both the accumulated-error correction and the residual-related bias are reduced. This explains why MARR can consistently improve perplexity by fine-tuning the residual contribution, while the gains on zero-shot accuracy may become more limited or task-dependent.

Unexplored Sensitivity of PID Gains. We uniformly set $K_p = K_i = K_d = 1.0$ following common practice of prior PID-in-deep-learning work [2, 25]. A systematic sensitivity sweep over these gains is costly under our PTQ pipeline, and we leave it to future work.

J Additional Iteration Trajectories

Figures 5–10 show the update trajectories of α_m and module-level reconstruction MSE on 30 randomly selected modules from Llama2-7b (Figures 5 and 6), 30 from Llama3-8b (Figures 7 and 8), and 30 from Llama2-13b (Figures 9 and 10), respectively, under W2A4. Across all 90 sampled modules, the reconstruction MSE drops substantially within the first one or two update steps and then stabilizes, while α_m converges to a module-specific value rather than a shared global one, supporting the choice of $T = 3$ and the necessity of module-adaptive scaling. The fast and stable convergence is a direct consequence of the tanh-bounded deviation signal in Eq. (11), which saturates as $J_m(\alpha_m)$ approaches its module-specific minimum, together with the K_d second-difference term in Eq. (12) that suppresses oscillation around the optimum.

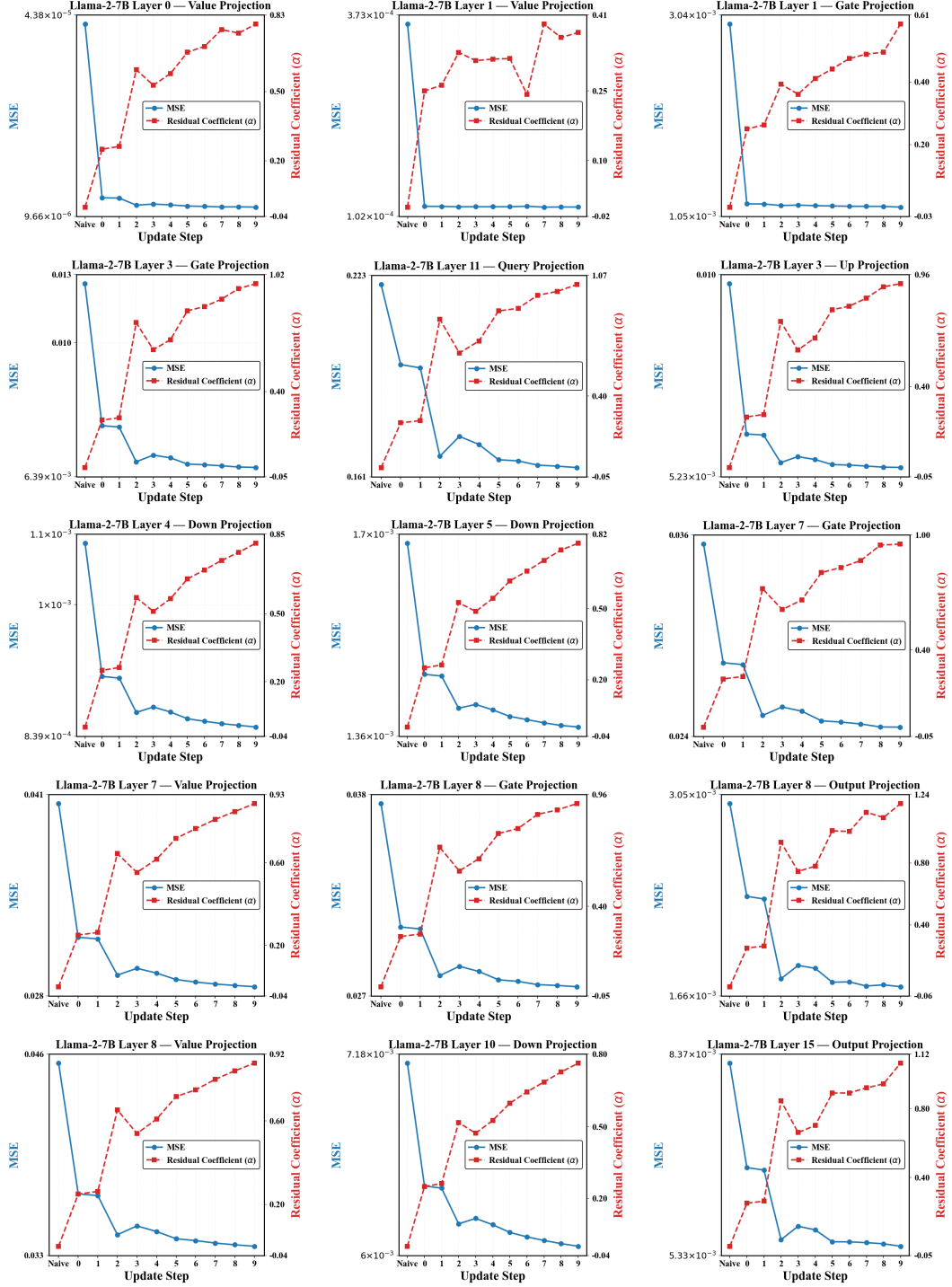


Figure 5: Update step trajectories of α_m and module-level reconstruction MSE on 15 randomly selected modules of Llama2-7b under W2A4 (Part 1/2).

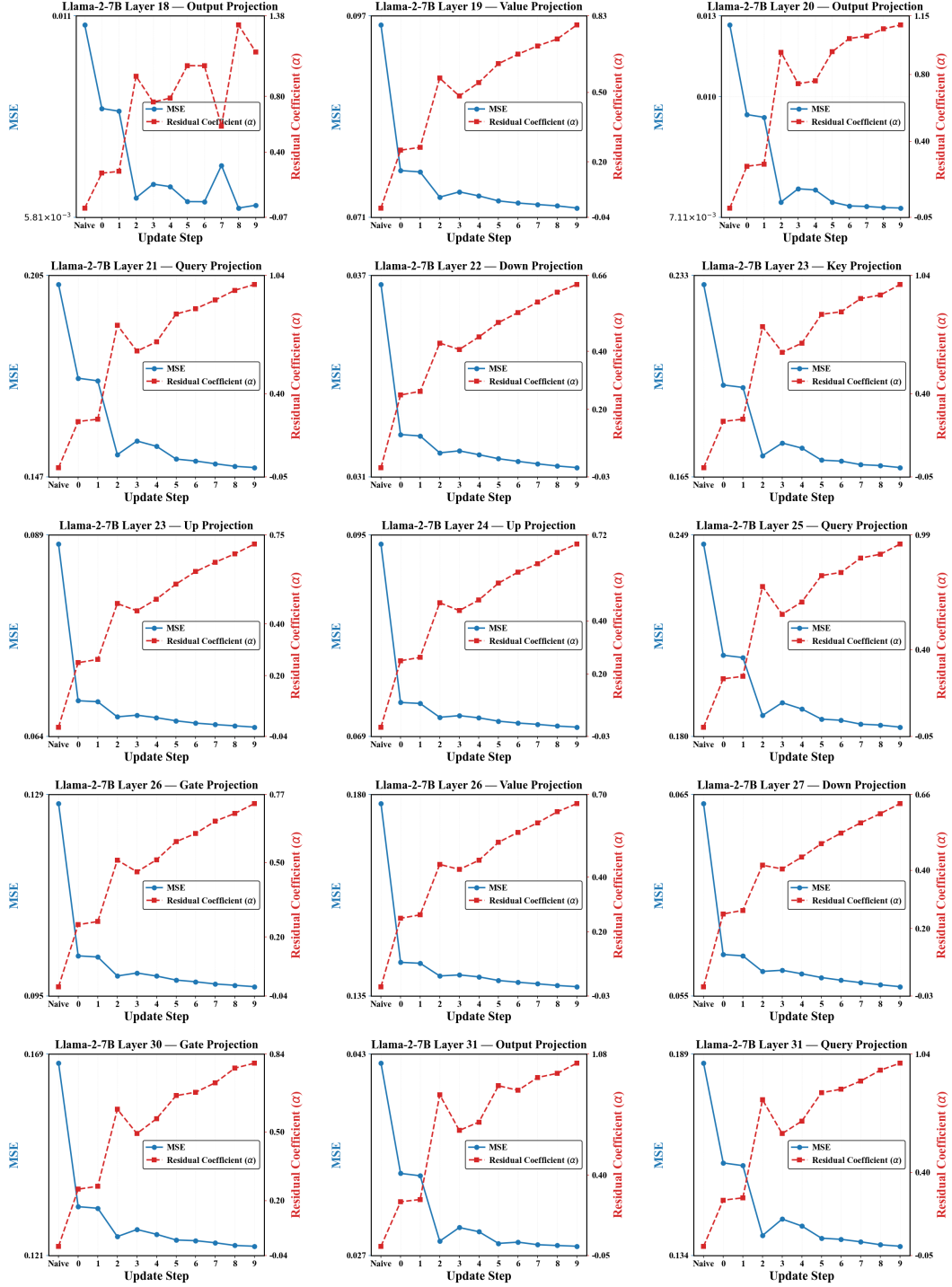


Figure 6: Update step trajectories of α_m and module-level reconstruction MSE on another 15 randomly selected modules of Llama2-7b under W2A4 (Part 2/2).

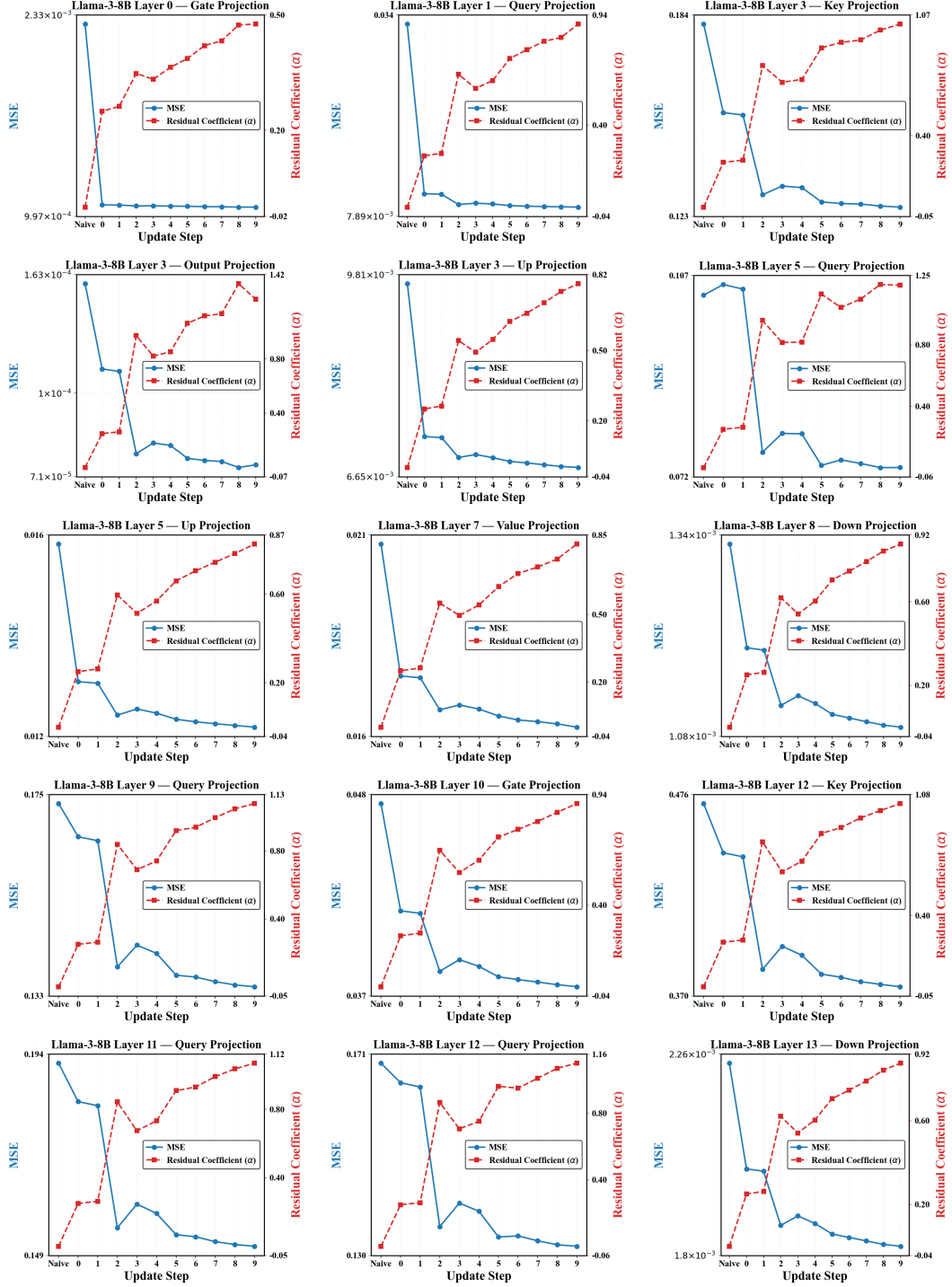


Figure 7: Update step trajectories of α_m and module-level reconstruction MSE on 15 randomly selected modules of Llama3-8b under W2A4 (Part 1/2).

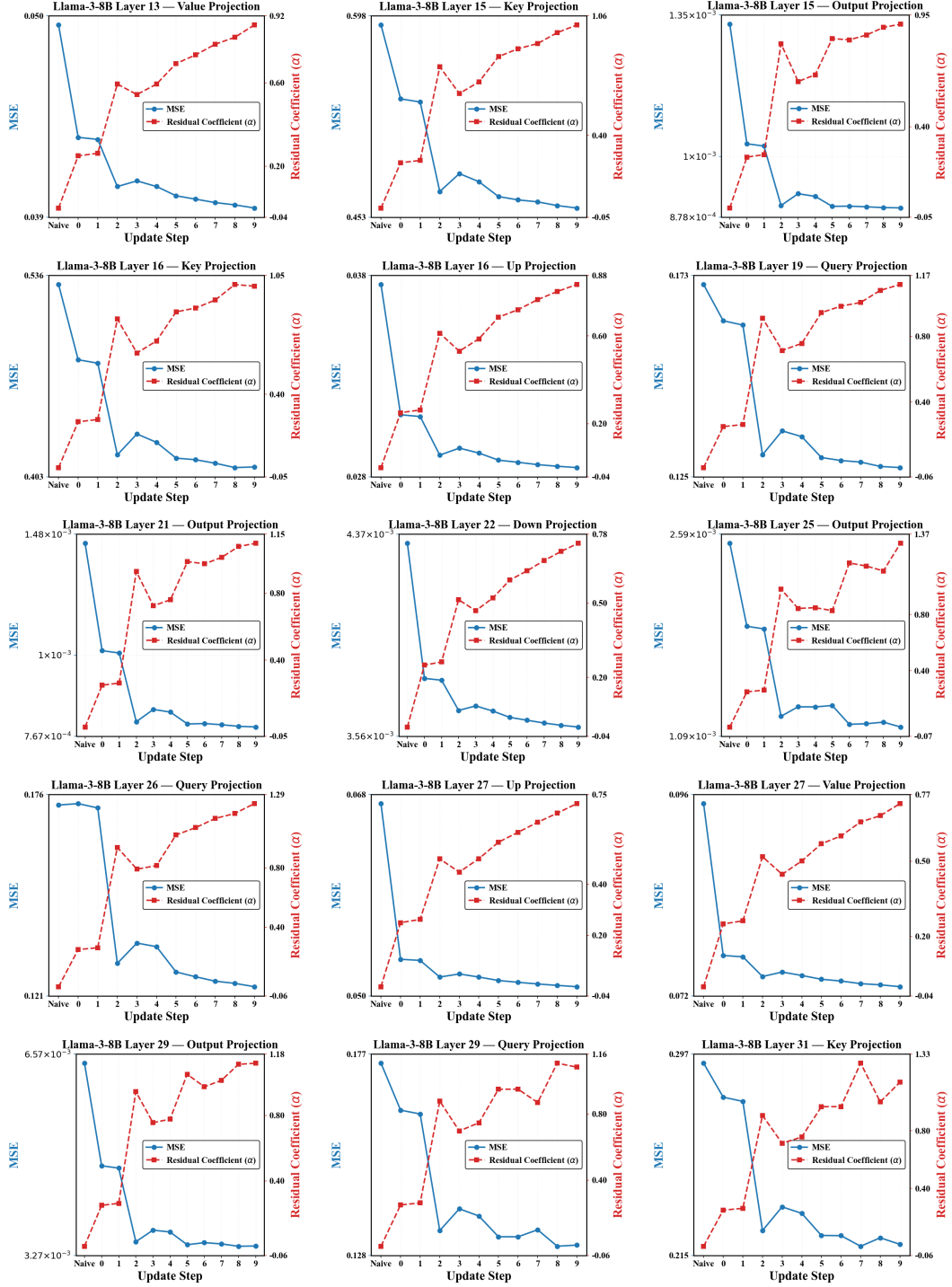


Figure 8: Update step trajectories of α_m and module-level reconstruction MSE on another 15 randomly selected modules of Llama3-8b under W2A4 (Part 2/2).

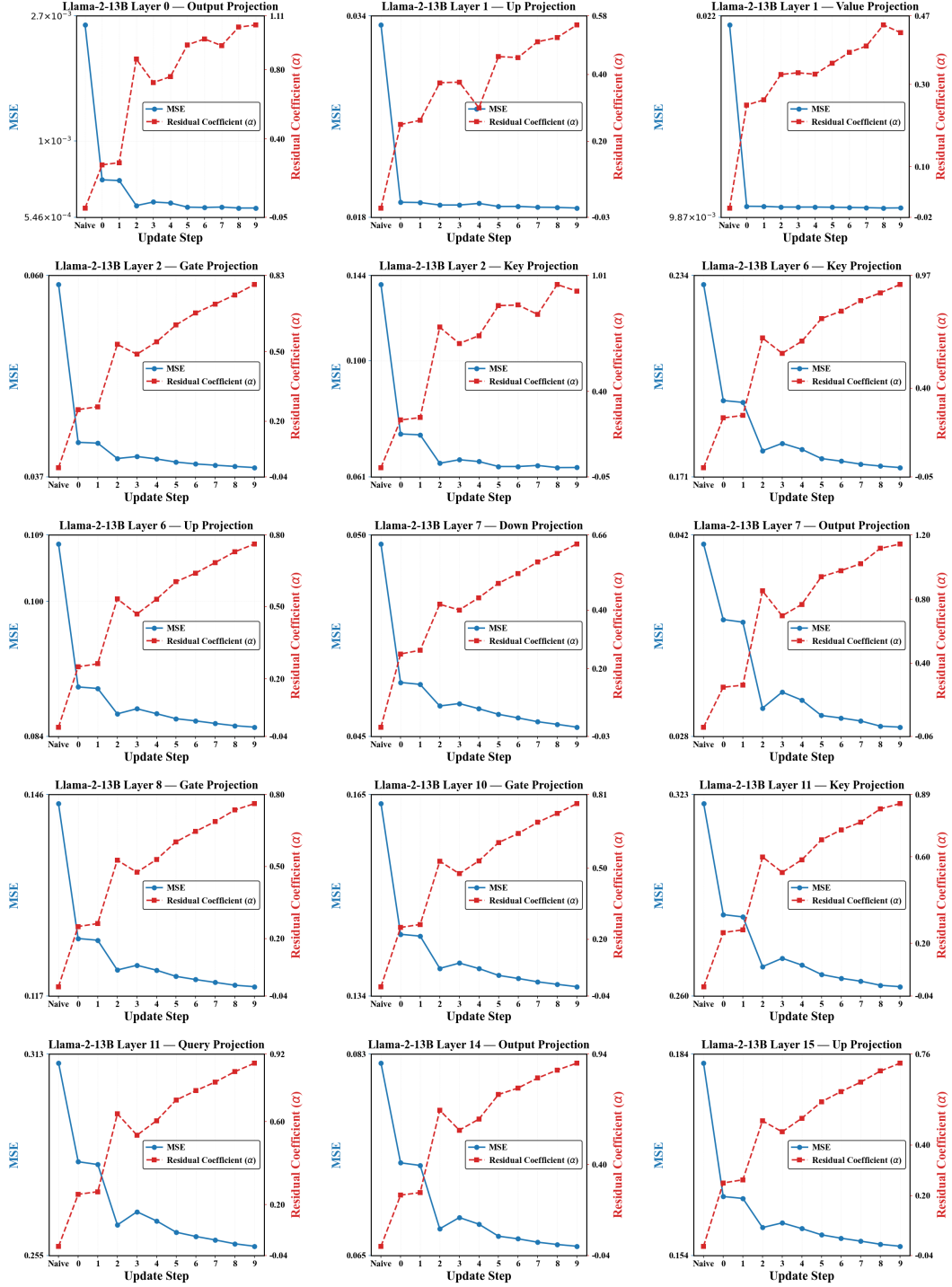


Figure 9: Update step trajectories of α_m and module-level reconstruction MSE on 15 randomly selected modules of Llama2-13b under W2A4 (Part 1/2).

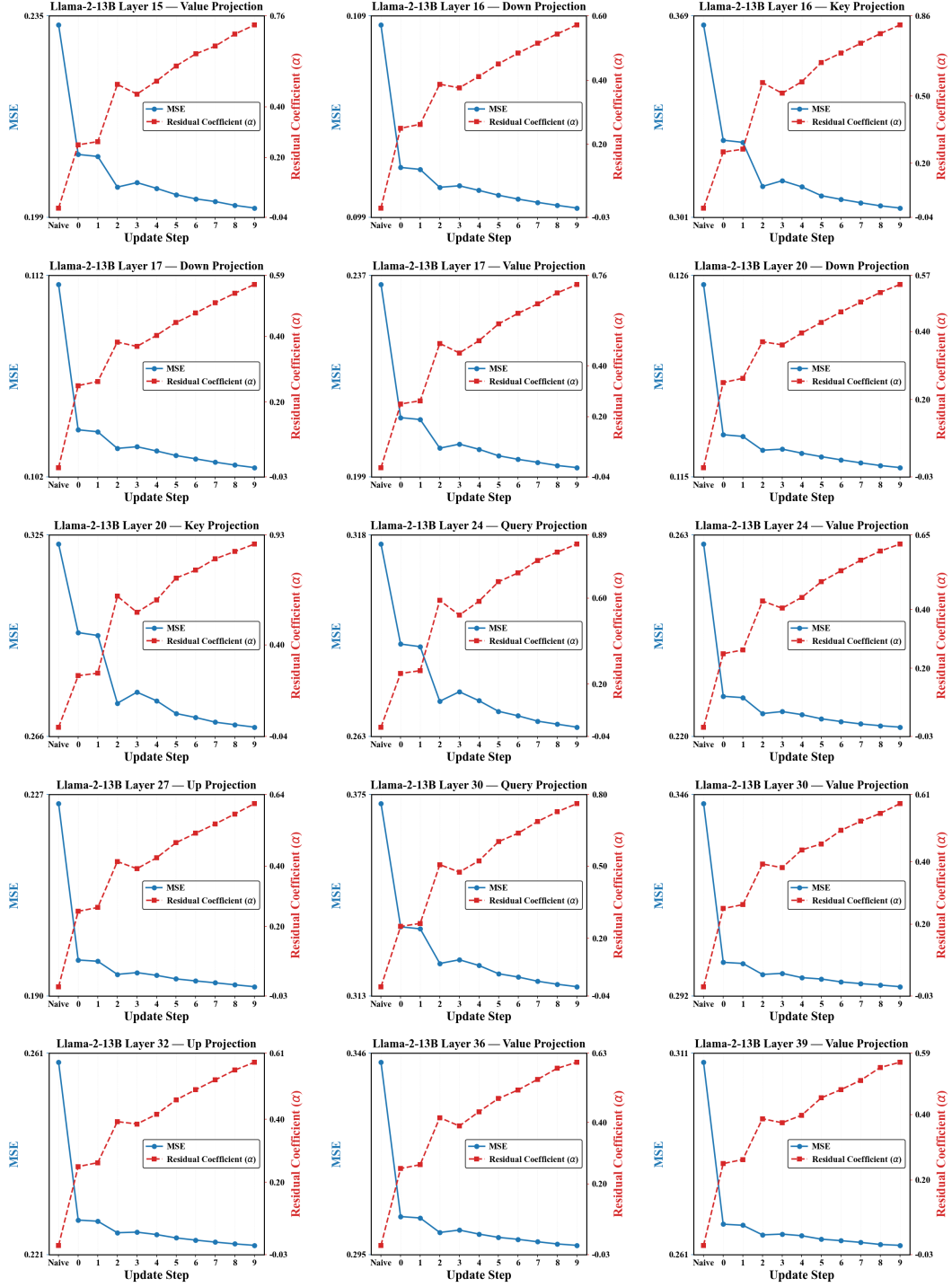


Figure 10: Update step trajectories of α_m and module-level reconstruction MSE on another 15 randomly selected modules of Llama2-13b under W2A4 (Part 2/2).